

GRADUAÇÃO

 FIAP

MOBILE APPLICATION DEVELOPMENT

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS



AULA 04

React Native

React Native: escreva uma vez, rode em qualquer lugar

“It is pointless to do with more what can be done with fewer.”

— William of Ockham

O QUE É REACT NATIVE

- Framework **cross-platform** para desenvolvimento mobile.
(Também é possível rodar em Web e outras plataformas, mas neste curso focaremos apenas em Android e iOS.)
- Permite escrever **uma vez** e rodar em **Android e iOS**.
- Baseado em **TypeScript** e componentes nativos.
(Embora suporte JavaScript, neste curso usaremos apenas TypeScript.)

REACT NATIVE E REACT WEB

- Compartilha conceitos de React: **componentes, props e state**.
- Diferença: não usamos HTML, usamos **componentes nativos**:
 - `View` em vez de `div`
 - `Text` em vez de `p/span`
 - `Image` para imagens
- A lógica de construção de UI é similar à do React Web.

CRIANDO UM APP

Vamos criar nosso primeiro projeto usando o **React Native Community CLI**:

PASSO 1: CRIAR O PROJETO

```
npx @react-native-community/cli@latest init FirstApp
```

PASSO 2: INICIE O METRO

Metro é a ferramenta de construção da aplicação para React Native.

```
npm start
```

PASSO 3: INICIE A APLICAÇÃO

```
npm run android
```

👉 Para rodar este projeto é necessário o Android Studio/Xcode. Sugestão: usar laboratório da FIAP.

ESTRUTURA DO PROJETO

Após criar um projeto com o Community CLI, você terá algo como:

DIRETÓRIOS

```
tests/      → pastas para testes unitários
.bundle/    → arquivos de build temporários
|
android/    → código nativo Android
ios/        → código nativo iOS
node_modules/ → dependências do projeto
```


ESTRUTURA DO PROJETO

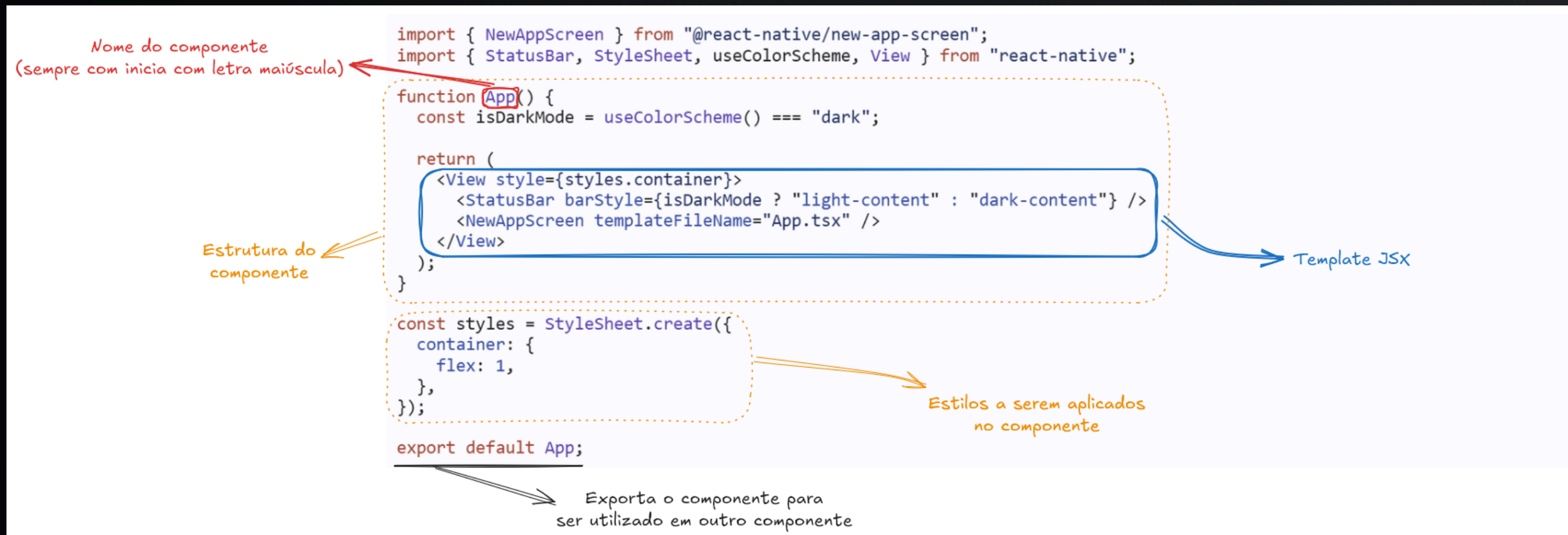
ARQUIVOS DE CONFIGURAÇÃO

<code>.eslintrc.js</code>	→ regras de linting do projeto
<code>.prettierrc.js</code>	→ configuração do Prettier
<code>.watchmanconfig</code>	→ configurações do Watchman (ferramenta de observação de arquivos)
<code>app.json</code>	→ configuração geral do app (nome, versão, expo settings)
<code>babel.config.js</code>	→ configuração do Babel (transpiler)
<code>index.js</code>	→ ponto de entrada principal do app
<code>metro.config.js</code>	→ configuração do Metro Bundler
<code>package.json</code>	→ dependências e scripts do projeto
<code>tsconfig.json</code>	→ configuração do TypeScript

ARQUIVOS PRINCIPAIS DE CÓDIGO

<code>App.tsx</code>	→ componente raiz do app, ponto inicial de desenvolvimento
<code>README.md</code>	→ documentação inicial do projeto
<code>Gemfile</code>	→ dependências Ruby (usadas para iOS)

App.tsx



COMO O REACT NATIVE FUNCIONA

- Todo o seu código é escrito em **TypeScript**, que é transpilado para **JavaScript puro**.
- Esse código é **empacotado (bundle)** em um único arquivo, enviado para o smartphone.
- No dispositivo, o React Native utiliza a **JSI (JavaScript Interface)** para comunicar seu app com o código nativo.
- Os **componentes React Native** são convertidos em **componentes nativos**, enquanto a lógica permanece em JavaScript.

DESENVOLVENDO COM O REACT NATIVE

- Durante o desenvolvimento, você mantém um **servidor de desenvolvimento** rodando, que faz esse processo automaticamente.
- Permite testar o app diretamente no dispositivo ou emulador sem recompilar tudo manualmente.
- O React Native conta com **Fast Refresh**, que atualiza o app instantaneamente ao salvar alterações, sem rebuild completo.

ESTILOS NO REACT NATIVE

- **Sem CSS tradicional:** os estilos são definidos em **JavaScript** usando objetos (`StyleSheet`) ou **inline styles**.
- A sintaxe lembra CSS, mas tudo é **JS objects**:

```
const styles = StyleSheet.create({  
  container: { flex: 1, justifyContent: "center", alignItems: "center" },  
  text: { color: "blue", fontSize: 20 },  
});
```

- **Flexbox** é padrão para layout, controlando alinhamento, distribuição e tamanho dos elementos.
- É possível controlar **cores, margens, padding, fontes e outros aspectos visuais** diretamente nos objetos de estilo.



ATUALIZANDO O APLICATIVO

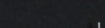
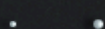
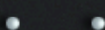
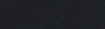
Vamos testar o **Fast Refresh**:

1. Abra o arquivo `App.tsx`.

2. Faça alterações no **texto, cores ou layout**.

3. Salve o arquivo e veja o app **atualizar automaticamente** no seu dispositivo ou emulador.

💡 Dica: você não precisa rebuildar o app. O **Fast Refresh** mantém o estado sempre que possível.



EXPO

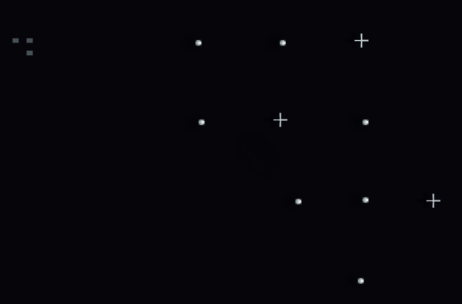


? POR QUE USAR UM FRAMEWORK?

- 🚀 Facilita o início rápido no desenvolvimento mobile
- 🛠️ Fornece ferramentas, bibliotecas e convenções integradas
- ⚙️ Evita configurações complexas de ambiente nativo no começo

EXPO GO PARA APRENDER

- Teste apps direto no celular sem precisar de build nativo
- Ideal para iniciantes: foco no **TypeScript**
-  Atualizações instantâneas (Fast Refresh)



ECOSSISTEMA EXPO

- SDK rico com APIs (câmera, notificações, sensores)
 - ⚡ Ferramentas de build e deploy simplificadas
 - 👥 Comunidade ativa e manutenção confiável
- 
- 

CRIANDO UM APP EXPO

```
npx create-expo-app@latest app-starter --template
```

👉 Selecione o template **Blank (TypeScript)**

```
cd app-starter  
npx expo start
```

📱 Escaneie o **QR Code** no celular com o **Expo Go**.

ESTRUTURA DE ARQUIVOS DO EXPO

O **Expo simplifica o desenvolvimento** mantendo apenas os arquivos essenciais, tornando o projeto mais limpo e fácil de navegar.

 .expo/	→ configuração interna do Expo e cache
 assets/	→ imagens, fontes e outros recursos
 node_modules/	→ dependências do projeto
 .gitignore	→ arquivos e pastas ignoradas pelo Git
 app.json	→ arquivo principal de configuração do app
 App.tsx	→ componente raiz do aplicativo
 index.ts	→ ponto de entrada do app
 package.json	→ dependências e scripts
 package-lock.json	→ lock das dependências
 tsconfig.json	→ configuração do TypeScript

Dica:

- Pasta `.expo/` e arquivos de configuração são **gerenciados pelo Expo**, você normalmente não precisa editá-los.
- Coloque seus recursos estáticos (imagens, fontes) em `assets/` para manter o projeto organizado.



ATUALIZANDO O APLICATIVO

Vamos personalizar nosso app inicial:

1. Abra o arquivo `App.tsx`.
2. Modifique a mensagem exibida na tela.
3. Altere a cor de fundo do app.
4. Ajuste a formatação do texto, como cor e tamanho.
5. Salve o arquivo e observe a **atualização automática** no dispositivo ou emulador.

LINKS DE REFERÊNCIAS

- [React Native](#)
- [Expo](#)
- [Expo Docs](#)

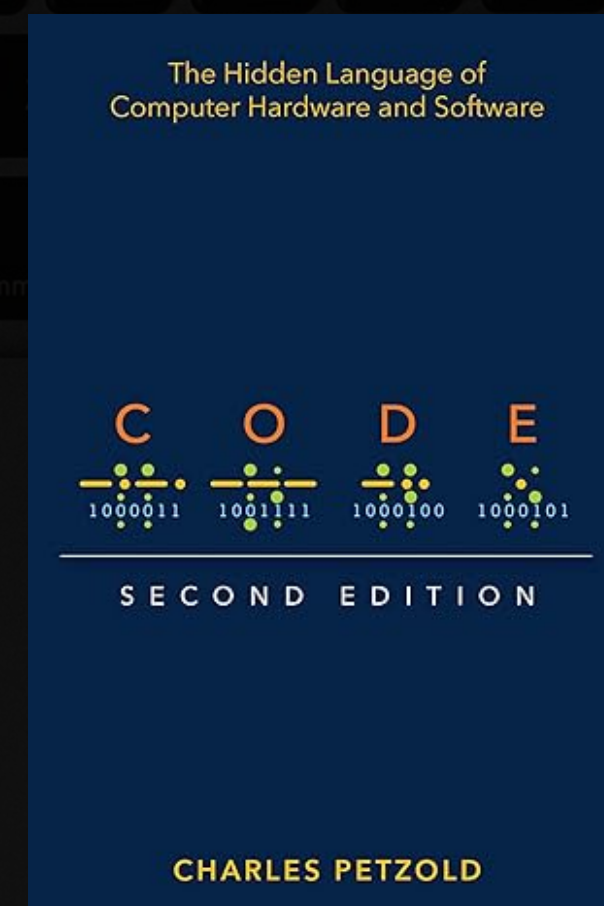
RECOMENDAÇÃO DE LEITURA

Se você quer entender a **base da computação** de forma clara e divertida, vale muito a pena ler o livro:

C.O.D.E – Charles Petzold

✨ Por que ler?

-  Explica como computadores realmente funcionam
-  Constrói o raciocínio desde conceitos simples até lógica digital
-  Mostra a conexão entre eletricidade, linguagem e programação
-  Ótimo para fortalecer fundamentos em tecnologia



LICENÇA

Este material é disponibilizado sob a licença Creative Commons Atribuição 4.0 Internacional (CC BY 4.0).

  Atribuição 4.0 Internacional

Autor: Felipe Cavalcante Vilella de Oliveira, 2025